

ABSTRACT

The rapid growth of Internet of Things (IoT) technology has transformed modern living by making homes smarter, safer, and more energy efficient. With increasing dependence on electronic appliances, manual control becomes inconvenient and inefficient. This project proposes an IoT-based home automation system that enables users to monitor and control household devices remotely through the internet.

The system uses sensors, a microcontroller, and a wireless communication module to collect realtime data and execute commands. Appliances such as lights, fans, and security systems can be controlled through a mobile application or web interface. The system also provides automation features like scheduled operation and energy monitoring, reducing unnecessary power consumption.

In addition, safety modules such as gas leak detection, fire alerts, and intrusion detection enhance household security by sending instant notifications to the user. The integration of IoT ensures continuous connectivity, remote accessibility, and efficient management of home resources.

The aim of this project is to design a reliable, cost-effective, and user-friendly smart home system that improves comfort, safety, and energy efficiency. This work demonstrates how IoT technology can be effectively implemented to build intelligent living environments for the future.

In addition to convenience, the system also focuses on improving home security by incorporating sensors such as motion detectors, temperature sensors, and gas leakage detectors. These sensors can trigger alerts or notifications to the user in case of abnormal conditions, thereby preventing potential hazards. The automation system can also be expanded with features like voice control and artificial intelligence to further enhance user experience.

One of the major advantages of this IoT-based home automation system is its scalability and cost-effectiveness. Compared to traditional automation systems, this solution is relatively affordable and easy to install, making it accessible to a wider range of users. Moreover, the system contributes to energy conservation by optimizing the usage of electrical devices, which is beneficial both economically and environmentally.

In conclusion, the proposed IoT-based home automation system demonstrates a practical and efficient approach to modern living. It provides a seamless integration of technology into daily life, offering improved comfort, safety, and energy management. As IoT continues to evolve, such smart systems are expected to become an essential part of future homes, paving the way for smarter and more connected living environments.

Contents

S.NO	TITLE	PAGE – NO
	1. INTRODUCTION	1
1.1	INTRODUCTION TO IOT AUTOMATION	2
1.2	FEATURES OF IOT SYSTEM	2
1.3	CHARACTERISTICS OF IOT SYSTEM	3
1.4	APPLICATIONS OF IOT SYSYTEM	4
	2. BLOCKDIAGRAM	5
	3. HARDWARE DESCRIPTION	6
3.1	ESP32 CONTROLLER	7
3.1.1	OPERATION WITH PINS	8
3.2	POWER SUPPLY	13
3.2.1	TRANSFORMER	14
3.2.2	RECTIFIER	16
3.2.3	FILTER CAPACITOR	17
3.2.4	VOLTAGE REGULATOR	19
3.3	RELAY MODULE	22
3.3.1	TYPES OF RELAY	22
3.3.2	WORKING OF RELAY	23
3.3.3	FEATURES	26
3.3.4	APPLICATIONS	28
3.4	L293D MOTOR DRIVER	30
3.4.1	PIN DIAGRAM OF L293D	30
3.4.2	CIRCUIT DIAGRAM OF L293D	32

3.4.3	APPLICATIONS	33
3.5	DC MOTOR	33
3.5.1	TYPES OF DC MOTOR	33
3.5.2	WORKING OF DC MOTOR	34
3.5.3	ADVANTAGES	37
3.6	BATTERY	38
3.6.1	WORKING	38
3.6.2	SPECIFICATIONS	39
3.6.3	APPLICATIONS	40
4	SOFTWARE EXPLANATION	41
4.1	INTRODUCTION	41
4.2	IOT INTERFACE	42
4.3	DESIGN CONSIDERATIONS	43
4.4	ESP32 PROGRAMMING USING ARDUINO IDE	44
5	PROGRAMMING OF DEVICE	48
6	ADVANTAGES AND APPLICATIONS	64
7	EXPLANATION OF RESULT	65
8	CONCLUSION AND FUTURESCOPE	67
9	REFERNCES	68

List of Figures

S.NO	FIGURE NAME	PAGE - NO
Fig 2.1	Block Diagram	5
Fig 3.1	Hardware image	7
Fig 3.2	Back side of module	8
Fig 3.3	pin explanation	14
Fig 3.4	Power supply	15
Fig 3.5	Transformer	15
Fig 3.6	Center tapped transformer	15
Fig 3.7	Half wave rectifier	16
Fig 3.8	Full wave rectifier	17
Fig 3.9	Bridge Rectifier	17
Fig 3.10	Bridge rectifier with center tapped transformer	18

Fig 3.11	Filter capacitor	19
Fig 3.12	Voltage regulator	20
Fig 3.13	Circuit diagram of power supply	20
Fig 3.14	Specifications of IC7805	21
Fig 3.15	Circuit symbol of relay	24
Fig 3.16	Relay Operation and use of protection diodes	25
Fig 3.17	Pin Diagram of L293D	30
Fig 3.18	Circuit Diagram of L293D	32
Fig 3.19	block Diagram of DC Motor	35
Fig 3.20	3.7V Battery	39
Fig 4.1	Tool bar necessary for the interface	41
Fig 7.1	Hardware kit	65
Fig 7.2	Working Model of Home Automation Using IoT	66

CHAPTER-1

INTRODUCTION

The Internet of Things (IoT) is an arrangement of interrelated computing gadgets, mechanical and digital machines, objects, animals or individuals that are given one kind of an identifiers and the capacity to exchange information over a system without requiring human-to-human or human-to-PC communication. IoT is a new concept that has evolved from the convergence of wireless technologies. Wireless communication is the transfer of information or signal between two or more points that are not connected by an electrical conductor. In IoT devices equipped with Wi-Fi allow the machine-to-machine communication. 1978. In the early years, the technology was not yet operational, due to an insufficient number of satellites orbiting the earth. On Jan. 17, 1994, after years of gradual growth, the final of the first 24 satellites was launched, and the GPS system was considered fully operational. Early GPS was designed primarily only for military but in 1996, President Bill Clinton determined that the system would be an asset to civilians as well as the military. This policy change made GPS technology available to the average individual, including fleet managers, who could see the benefit of using the technology to keep tabs on their vehicles. In the early days of fleet tracking, in order to properly track a fleet, each vehicle had to be enabled with a costly GPS.

Nowadays, there is an increase in the number of accidents that happen in the world. As the population is increasing, there is the number of cars increasing on the road that contributes to severe accidents that happen daily. Around 80 per cent of accidents contribute to the loss of many lives. Mostly, the growing countries are being targeted by the day to day road accidents. The major reason is the lack of infrastructure, lack of traffic control and accident management. Out of all the developing countries, India has been listed as the country with a higher number of accidents .

The most prominent reason for the loss of a life during an accident is the unavailability of immediate help that can save a person's life by a few seconds. The moment an accident has occurred, the life of all passengers travelling in the vehicle is at stake. It all depends on response time that can save their lives by a few minutes or seconds. According to the statistics, reducing accident delay time by even 1 minute can save 6 per cent of lives. Hence, this response time is very crucial, and it needs to be reduced or at least either improved to save their lives .

To contribute to our society and reduce the number of accidents happening in our day to day life, there are several techniques and mechanisms that can drop down the rate of accidents and

can save lot lives. Living in a tech world that is growing day by day with new technologies, we can apply these techniques in our society and help them overcome such problems. The Vision of the Internet of Things (IoT) has come out to reach unexpected bounds of today's computing world. It is a concept that not only can impact human's life but also how they function . The heart of IOT is smart sensors without which it would not have existed. These sensors form a vast network for their communication.

1.1 INTRODUCTION TO IOT AUTOMATION

The **Internet of Things (IoT)** enables physical devices to connect, communicate, and exchange data over the internet. IoT automation enhances this capability by allowing systems to operate with minimal human intervention. By integrating sensors, actuators, and cloud-based platforms, IoT solutions can monitor real-time conditions and trigger automated responses. This leads to improved efficiency, accuracy, and decision-making in various applications. IoT automation also supports remote control and predictive maintenance, making systems smarter and more reliable. Overall, it forms a foundational technology for modern smart environments and intelligent systems.

1.2 Features of IOT SYSTEM

Features of IOT Systems are as follows:

1. Connectivity

IoT devices must connect to each other and to the cloud through Wi-Fi, Bluetooth, cellular networks, or other communication protocols.

2. Sensors & Data Collection

Sensors gather real-time data from the environment (such as temperature, motion, humidity) and send it for processing.

3. Automation & Intelligence

IoT systems can make decisions automatically based on predefined rules or AI/ML algorithms, reducing human involvement.

4. Data Processing & Analytics

Data collected by devices is processed locally (edge computing) or in the cloud to extract useful insights.

5. Remote Access & Control

Users can monitor, manage, and control IoT devices from anywhere using mobile apps or web dashboards.

1.3 Characteristics of IOT SYSTEM

The main characteristics of IOT system, They often include the following additional characteristics:

- 1.All IoT devices can connect and communicate with each other, forming a shared network.
- 2.IoT systems continuously sense and collect data from the environment using sensors.
- 3.They can analyze data and automatically make decisions or trigger actions without human input.
- 4.IoT systems can be easily expanded by adding more devices without affecting performance.
- 5.Devices can adapt to changes in their environment in real time and update their operations accordingly.

1.4 APPLICATIONS OF IOT SYSTEM

1. Smart Homes

Used for home automation such as smart lighting, smart thermostats, security systems, and appliance control.

2. Healthcare & Wearables

IoT devices track patient health, monitor vital signs, and support remote healthcare services.

3. Industrial Automation (IIoT)

Factories use IoT for machine monitoring, predictive maintenance, quality control, and improved production efficiency.

4. Smart Agriculture

IoT sensors help monitor soil moisture, crop health, irrigation, temperature, and automate farming processes.

5. Smart Cities

Applications include smart traffic control, waste management, street lighting, and environmental monitoring.

6. Transportation & Logistics

Used for fleet tracking, vehicle monitoring, route optimization, and real-time shipment tracking.

7. Retail & Supply Chain

IoT enables inventory tracking, automated checkout, smart shelves, and improved supply chain management.

CHAPTER-2

BLOCK DIAGRAM

1.1 BLOCK DIAGRAM OVERVIEW:

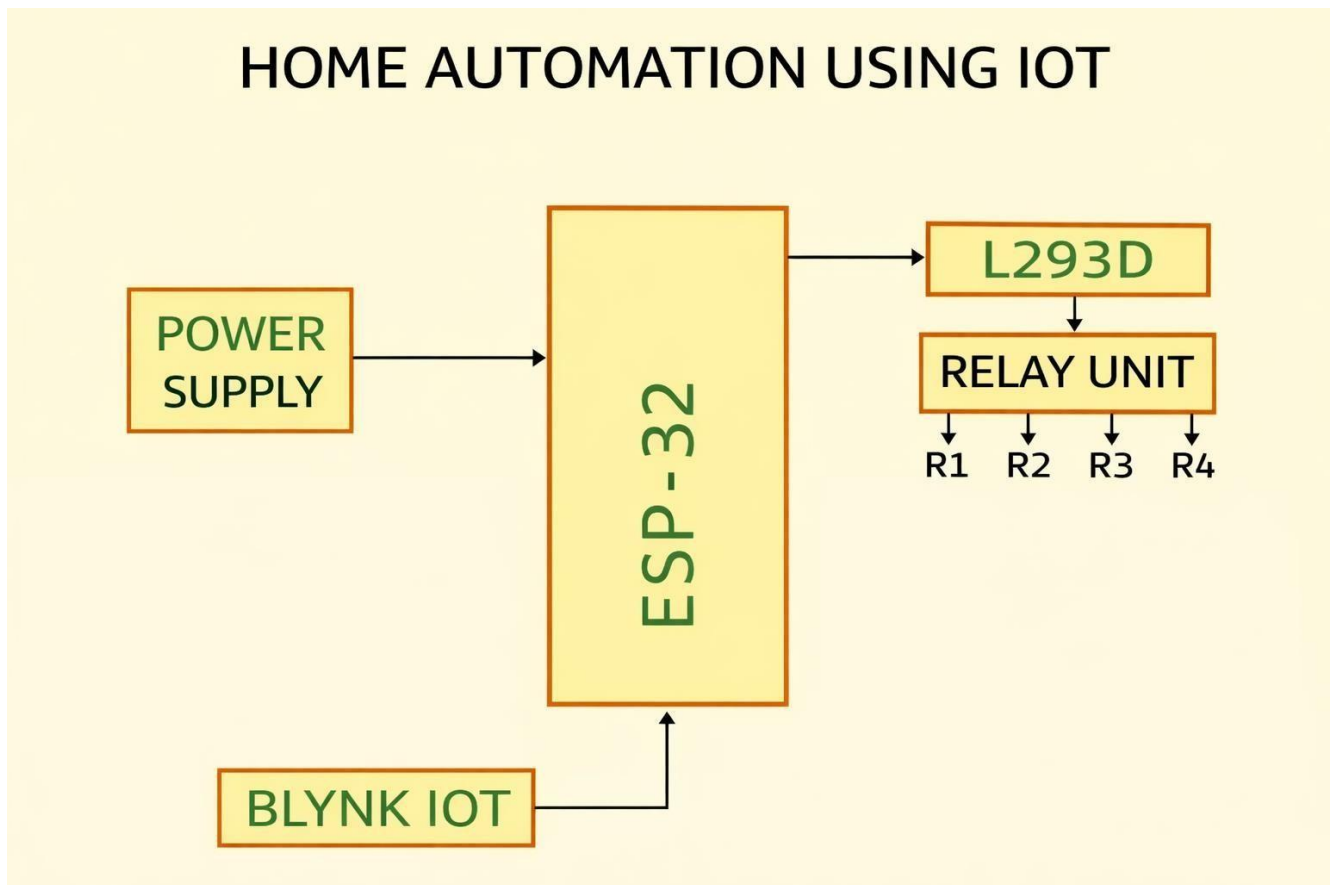


Fig.2.1 block diagram

CHAPTER-3

HARDWARE DESCRIPTION

The list of components used for designing of Accident Alert Vehicle System

Using GPS and GSM are

- ESP32 Controller
- Power supply
- Relay Module
- L293D Motor Driver
- DC motor
- Battery

3.1 ESP32 CONTROLLER

The **ESP32** is a highly integrated microcontroller chip designed by Espressif Systems, known for its excellent performance, rich features, and low cost, making it a popular choice for modern IoT applications. It comes with a **dual-core Tensilica LX6 processor** that can run at speeds up to 240 MHz, providing strong computational capability for multitasking and complex operations. One of the defining strengths of the ESP32 is its **built-in Wi-Fi (802.11 b/g/n)** and **Bluetooth/BLE** support, which enables seamless wireless communication without the need for external modules.

In addition to its processing power, the ESP32 includes a wide variety of hardware features such as **multiple GPIO pins**, **analog-to-digital converters (ADC)**, **digital-to-analog converters (DAC)**, **touch sensors**, **pulse counters**, and **PWM outputs**. It also supports hardware interfaces like **UART**, **SPI**, **I2C**, **CAN**, **I2S**, and **Ethernet**, giving it flexibility for interfacing with sensors, motors, displays, and other peripherals.

The chip is designed with **low power modes**, making it suitable for battery-powered IoT devices that require energy efficiency. It can operate in deep sleep with very low current consumption. The ESP32 is compatible with popular development environments like **Arduino IDE**, **MicroPython**, **ESP-IDF**, and **PlatformIO**, offering developers a wide range of tools and libraries.

Because of its high performance, wireless connectivity, and versatile peripheral support, the ESP32 is widely used for smart home devices, automation systems, sensor networks, wearable technology, robotics, and other embedded applications. Its strong community support and extensive documentation make it a reliable and beginner-friendly platform for IoT innovation.



Fig 3.1 HARDWARE IMAGE

3.1.1 OPERATION WITH PINS:

The ESP32 is a highly integrated microcontroller that operates as a dual-core system-on-chip designed for wireless communication and general-purpose embedded processing. At its heart, it uses two 32-bit Xtensa LX6 CPU cores, which typically run between 160 MHz and 240 MHz. These cores can operate independently under the FreeRTOS real-time operating system, allowing multitasking, where one core often handles Wi-Fi or Bluetooth protocol stacks while the other executes user application code. The device also contains an Ultra-Low-Power (ULP) co-processor that can run simple tasks during deep sleep to conserve energy, enabling extremely low-power applications such as battery-powered sensors.

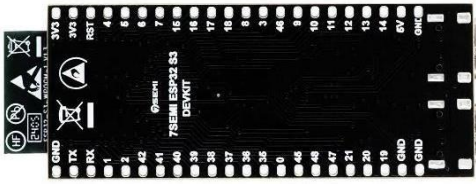


Fig.3.2 Back side of module.

When the ESP32 powers on, an internal ROM-based bootloader starts first, initializes basic system components, and reads the firmware stored in external SPI flash memory. This flash memory contains the partition table, application firmware, file system (such as SPIFFS or LittleFS), Wi-Fi calibration data, and other system information. The bootloader selects the correct partition, loads the main program into internal RAM, and then transfers control to the user application, which is managed under FreeRTOS. Once the system is running, tasks are scheduled based on priority, and each core can run one task at a time, providing true parallel execution for complex or time-sensitive applications.

The ESP32's wireless subsystem is one of its key strengths. It includes full 2.4 GHz Wi-Fi 802.11 b/g/n capability, which operates through an internal RF front end and MAC layer that manages authentication, beacon handling, packet transmission, and reception. On top of the Wi-Fi radio, the device includes a complete TCP/IP networking stack that supports protocols like TCP, UDP, IP, DHCP, HTTP, and MQTT. It can function as a Wi-Fi station, an access point, or even both simultaneously. In addition to Wi-Fi, the ESP32 contains Bluetooth Classic and Bluetooth Low Energy (BLE) capabilities, allowing connection with a wide variety of wireless devices. BLE can operate as a central or peripheral, supporting applications such as beacons, fitness sensors, or wireless controls.

To support its processing and wireless operation, the ESP32 includes a versatile memory architecture consisting of internal SRAM divided into instruction RAM, data RAM, and RTC memories. The real-time clock (RTC) domain stays powered during sleep modes and can preserve state or store small amounts of data. The device also supports memory-mapped SPI flash access, allowing executed code

to run directly from external flash. This combination of internal and external memory makes it possible to store large programs, web servers, graphics assets, and user files.

The ESP32 also integrates a wide variety of digital and analog peripherals that operate through a powerful internal IO-multiplexing system. Its general-purpose I/O pins can handle multiple functions such as UART, SPI, I²C, I²S (for audio), PWM outputs through the LEDC controller, and specialized functions like the RMT (Remote Control) module for IR modulation. Analog peripherals include a multi-channel 12-bit ADC, two 8-bit DACs, a Hall-effect sensor, and capacitive touch sensing inputs, enabling a large range of sensor interfaces without external components. Built-in timers, watchdogs, and hardware cryptographic accelerators further expand its capabilities and improve system reliability and security.

Power management is another central part of ESP32 operation. It offers several modes including active mode, modem-sleep, light-sleep, and deep-sleep. In deep-sleep mode, the main CPU cores shut down completely while the RTC and ULP co-processor remain active, reducing power consumption to microamp levels. The device can wake from deep sleep using timers, external interrupts on designated GPIOs, capacitive touch pads, or signals processed by the ULP. This design makes the ESP32 ideal for energy-efficient IoT devices that need intermittent connectivity.

Once the device is running, the FreeRTOS kernel manages the user program. Application code typically initializes wireless connections, configures peripherals, starts background tasks, and enters its main loop. Networking tasks may handle MQTT messaging, HTTP server responses, or cloud communication while other tasks read sensors, control actuators, or process data. Because the ESP32 includes built-in hardware security features such as AES, SHA, RSA acceleration, secure boot, and flash encryption, it can operate safely in networked environments and protect firmware and data from unauthorized access.

In summary, the ESP32 operates as a fully integrated, dual-core, wireless microcontroller that loads firmware from external flash, manages multitasking under FreeRTOS, and provides robust Wi-Fi and Bluetooth connectivity alongside extensive digital and analog peripherals. Its flexible power- management system and ULP co-processor allow it to operate efficiently in both high-performance .

1. Power Pins

→ 3.3V Pin

- Provides **regulated 3.3V output**.
- Can be used to power sensors, modules, etc. □ ESP32 operates internally at 3.3V.

→ VIN Pin

- Used when powering the board externally with **5V–12V**.
- Voltage is regulated down to 3.3V by the onboard regulator.

→ GND Pins

- Ground connection.
- Required to complete all circuits.

2. GPIO Pins (General Purpose I/O)

ESP32 has **34 GPIO pins**, but some have restrictions.

These pins can be used as:

- Digital Input (read HIGH/LOW)
- Digital Output (drive LEDs, relays, motors)
- PWM (brightness/motor speed control)
- I2C, SPI, UART functions
- Touch input
- ADC (analog input)

Input-Only Pins

Pins **GPIO34, 35, 36, 39**

- Can only read INPUT (cannot output HIGH/LOW) □
Best for sensors.

Pins with Boot Restrictions

These pins affect booting:

- **GPIO0** → Boot mode / flashing mode □ **GPIO2** → Must be HIGH during boot
- **GPIO15** → Should be LOW during boot

- **GPIO12** → Connect carefully; affects internal voltage

Avoid using these for relays or pulldown circuits unless you know what you're doing.

3. ADC Pins (Analog Input)

ESP32 has **18 ADC channels**.

ADC1: GPIO32 to GPIO39 ADC2: GPIO0, 2, 4, 12–15, 25–27

Used for reading:

- Temperature sensors
- Soil moisture sensors
- Potentiometers
- Light sensors

Note: ADC2 cannot be used when Wi-Fi is active.

4. DAC Pins (Analog Output)

ESP32 has **2 DAC channels**:

DAC Channel	Pin
DAC1	GPIO25
DAC2	GPIO26

- Generates **true analog voltage** (0–3.3V).
- Used for generating audio waves, analog signals, etc.

5. Touch Sensor Pins

ESP32 supports capacitive touch sensing on these pins:

GPIO 0, 2, 4, 12, 13, 14, 15, 27, 32, 33

Used for:

- Touch buttons

- Touch switches
- Human detection

6. Communication Pins

A. UART (Serial Communication)

- **TX0** → **GPIO1**
- **RX0** → **GPIO3**

Used for:

- Uploading code
- Serial monitor

Extra UARTs (UART1, UART2) can be mapped to any GPIO pins.

B. I2C Communication

Default pins (can be changed):

- **SDA** → **GPIO21**
- **SCL** → **GPIO22**

Used for:

- LCD displays
- Sensors (DHT12, MPU6050, BMP280, OLED)

C. SPI Communication

Default pins:

- **MOSI** → **GPIO23**
- **MISO** → **GPIO19**
- **SCK** → **GPIO18**
- **CS** → **GPIO5**

Used for:

- SD cards
- TFT displays
- Flash memory modules

7. PWM Pins

- All GPIO pins except **input-only pins** support PWM.
- Used for:
 - LED brightness control
 - Motor speed control
 - Buzzer tone control

PWM can be:

- Any frequency (1 Hz to 40 MHz)
- Custom duty cycle (0 – 255)

8. EN (Enable) Pin

- Acts as **reset pin**.
- Pulling EN **LOW** **restarts** the board.
- Pulling HIGH keeps the board on.

9. Boot Button

- Connected to **GPIO0**.
- Used for:
 - Flash mode when uploading code manually.
 - Can be used as a regular input button in projects.

10. Reset Button

- Restarts the ESP32.
- Doesn't erase code.

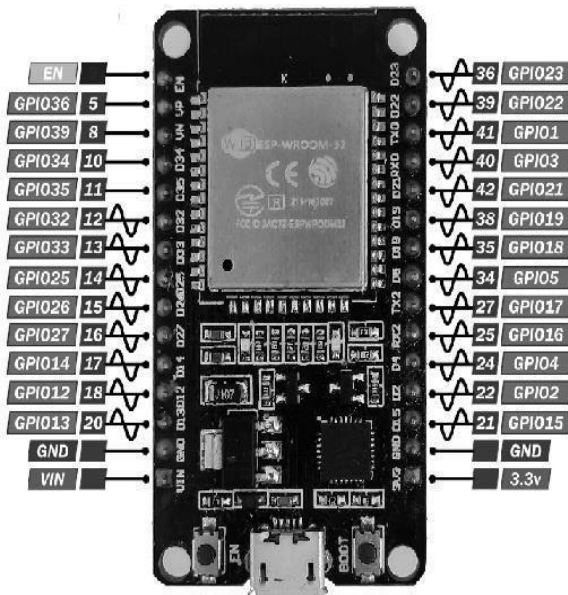


Fig 3.3 pin explanation

3.2 POWER SUPPLY:

All digital circuits require regulated power supply. In this article we are going to learn how to get a regulated positive supply from the mains supply.

ESP32 development board operates on **3.3V**, and its power supply system is designed to provide stable voltage to the microcontroller and connected components. It can be powered in two main ways: through the **5V USB port** or the **VIN pin** (5–12V), both of which pass through an onboard voltage regulator that converts the input voltage into a stable **3.3V**. The board also provides **3.3V output pins** to power external sensors and modules, along with multiple **GND pins** for grounding. This regulated power system ensures reliable performance and protects the ESP32 from voltage fluctuations. The ESP32 also supports lowpower operation modes, making it suitable for battery-powered IoT applications. It can run efficiently on lithium-ion batteries, power banks, or solar-based power systems. The onboard regulator ensures stable operation even when the input voltage fluctuates, while multiple sleep modes—such as deep sleep and light sleep—help reduce power consumption significantly. This makes the ESP32 ideal for longterm, energy-efficient projects where continuous operation is required without frequent charging or external power support.

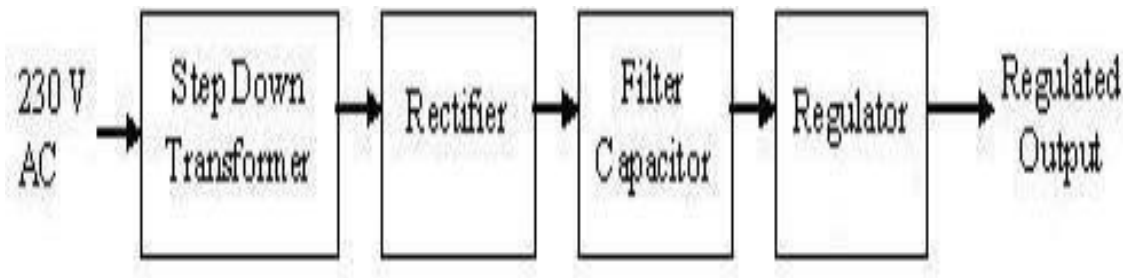


Fig:3.5 shows the basic block diagram of a fixed regulated power supply. Let us go through each block.

3.2.1 TRANSFORMER

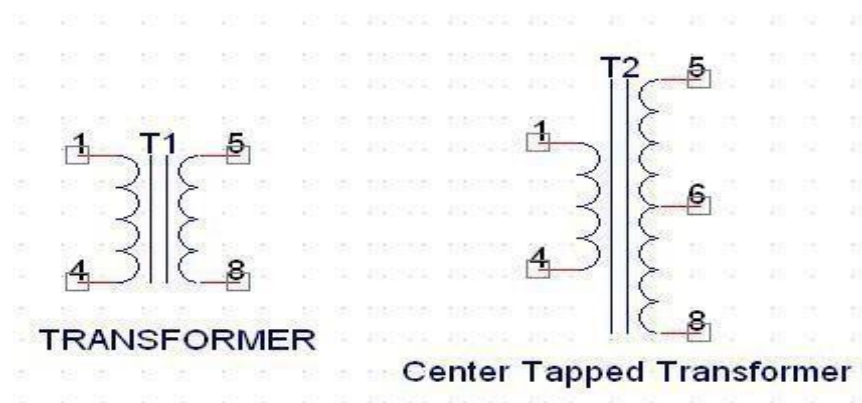


Fig 3.6

Fig 3.7

A transformer consists of two coils also called as “WINDINGS” namely PRIMARY & SECONDARY.

They are linked together through inductively coupled electrical conductors also called as CORE. A changing current in the primary causes a change in the Magnetic Field in the core & this in turn induces an alternating voltage in the secondary coil. If load is applied to the secondary then an alternating current will flow through the load. If we consider an ideal condition then all the energy from the primary circuit will be transferred to the secondary circuit through the magnetic field.

$$P_{\text{primary}} = P_{\text{secondary}}$$

So

$$I_p V_p = I_s V_s$$

The secondary voltage of the transformer depends on the number of turns in the Primary as well as in the secondary.

$$\frac{V_s}{V_p} = \frac{N_s}{N_p}$$

3.2.2 Rectifier

A rectifier is a device that converts an AC signal into DC signal. For rectification purpose we use a diode, a diode is a device that allows current to pass only in one direction i.e. when the anode of the diode is positive with respect to the cathode also called as forward biased condition & blocks current in the reversed biased condition.

Rectifier can be classified as follows:

1) Half Wave rectifier.

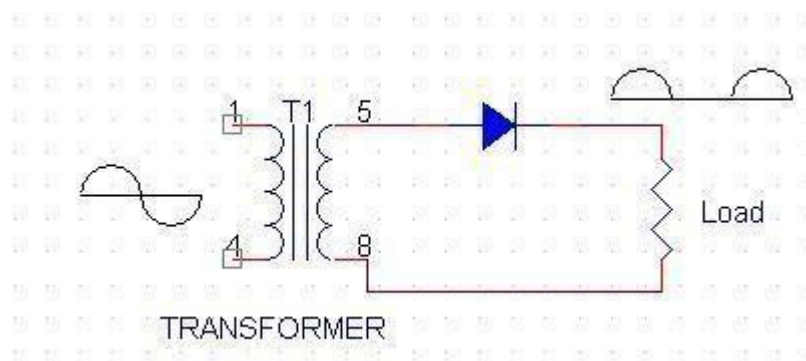


FIG 3.8

This is the simplest type of rectifier as you can see in the diagram a half wave rectifier consists of only one diode. When an AC signal is applied to it during the positive half cycle the diode is forward biased & current flows through it. But during the negative half cycle diode is reverse biased & no current flows through it. Since only one half of the input reaches the output, it is very inefficient to be used in power supplies.

2) Full wave rectifier.

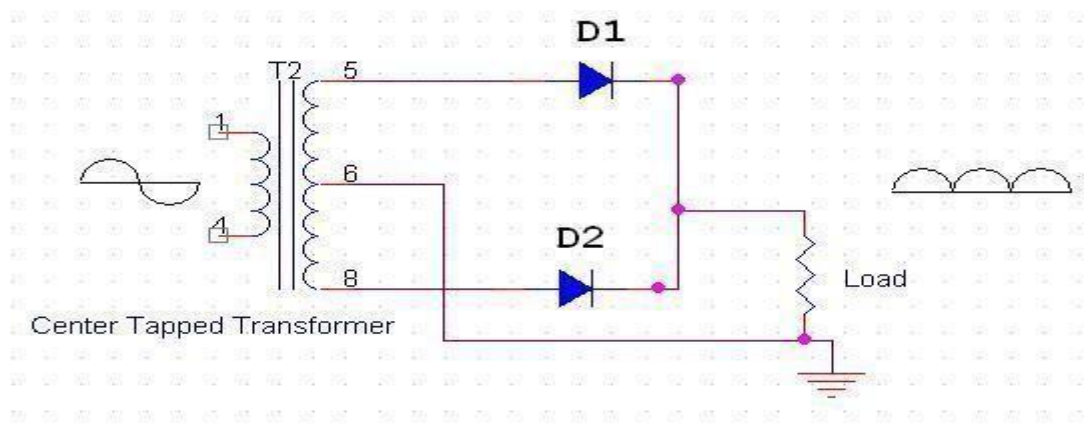


FIG 3.9

Half wave rectifier is quite simple but it is very inefficient, for greater efficiency we would like to use both the half cycles of the AC signal. This can be achieved by using a center tapped transformer i.e. we would have to double the size of secondary winding & provide connection to the center. So during the positive half cycle diode D1 conducts & D2 is in reverse biased condition. During the negative half cycle diode D2 conducts & D1 is reverse biased. Thus we get both the half cycles across the load

One of the disadvantages of Full Wave Rectifier design is the necessity of using a center tapped transformer, thus increasing the size & cost of the circuit. This can be avoided by using the Full Wave Bridge Rectifier.

3) Bridge rectifier

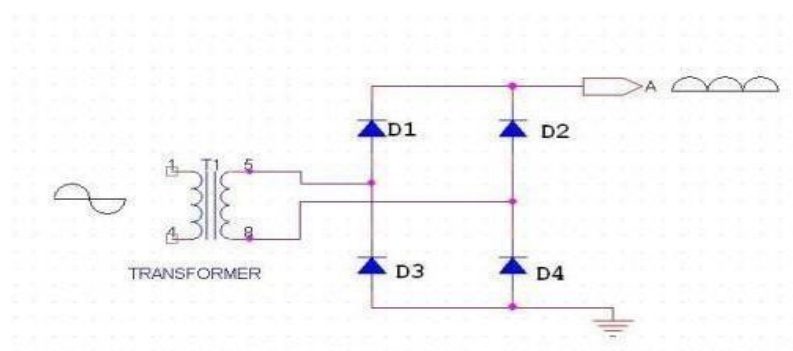
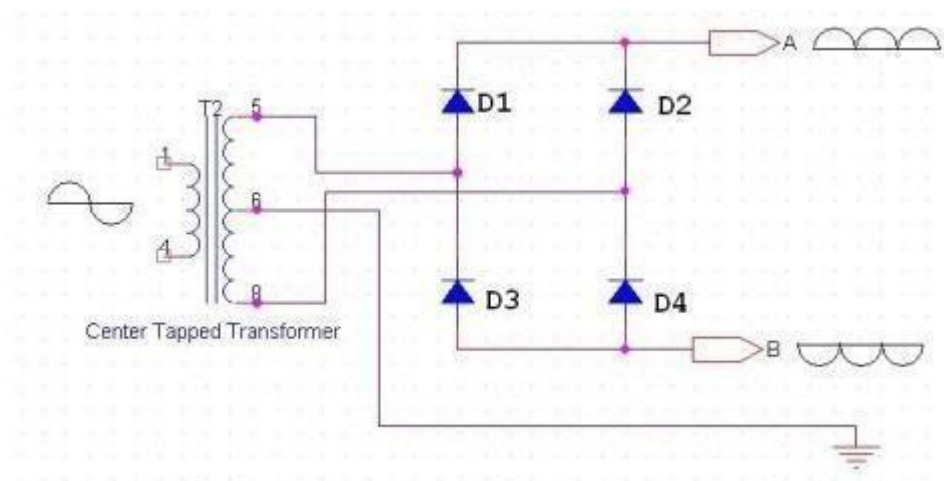


Fig 3.10

As the name suggests it converts the full wave i.e. both the positive & the negative half cycle into DC thus it is much more efficient than Half Wave Rectifier & that too without using a center tapped transformer thus much more cost effective than Full Wave Rectifier.

Full Bridge Wave Rectifier consists of four diodes namely D1, D2, D3 and D4. During the positive half cycle diodes D1 & D4 conduct whereas in the negative half cycle diodes D2 & D3 conduct thus the diodes keep switching the transformer connections so we get positive half cycles in the output.

**Fig 3.11 bridge rectifier with center tapped transformer**

If we use a center tapped transformer for a bridge rectifier we can get both positive & negative half cycles which can thus be used for generating fixed positive & fixed negative voltages.

3.2.3 FILTER CAPACITOR

Even though half wave & full wave rectifier give DC output, none of them provides a constant output voltage. For this we require to smoothen the waveform received from the rectifier. This can be done by using a capacitor at the output of the rectifier this capacitor is also called as “FILTER CAPACITOR” or “SMOOTHING CAPACITOR” or “RESERVOIR CAPACITOR”. Even after using this capacitor a small amount of ripple will remain.

We place the Filter Capacitor at the output of the rectifier the capacitor will charge to the peak voltage during each half cycle then will discharge its stored energy slowly through the load while the rectified voltage drops to zero, thus trying to keep the voltage as constant

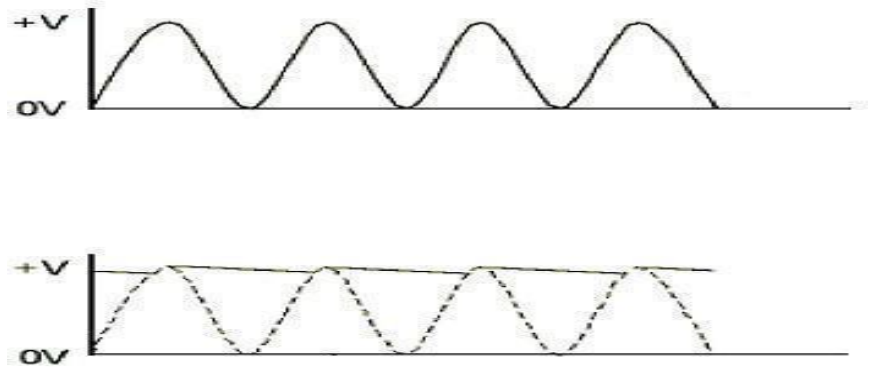


Fig 3.12

If we go on increasing the value of the filter capacitor then the Ripple will decrease. But then the costing will increase. The value of the Filter capacitor depends on the current consumed by the circuit, the frequency of the waveform & the accepted ripple.

$$C = \frac{V_r F}{I}$$

Where,

V_r = accepted ripple voltage.(should not be more than 10% of the voltage)

I = current consumed by the circuit in Amperes.

F = frequency of the waveform. A half wave rectifier has only one peak in one cycle so

$F=25\text{hz}$

Whereas a full wave rectifier has Two peaks in one cycle so $F=100\text{hz}$

3.2.4 VOLTAGE REGULATOR

A Voltage regulator is a device which converts varying input voltage into a constant regulated output voltage. Voltage regulator can be of two types

1) Linear Voltage Regulator

Also called as Resistive Voltage regulator because they dissipate the excessive voltage resistively as heat.

2) Switching Regulators.

They regulate the output voltage by switching the Current ON/OFF very rapidly. Since their output is either ON or OFF it dissipates very low power thus achieving higher efficiency as compared to linear voltage regulators. But they are more complex & generate high noise due to their switching action. For low level of output power switching regulators tend to be costly but for higher output wattage they are much cheaper than linear regulators.

The most commonly available Linear Positive Voltage Regulators are the 78XX series where the XX indicates the output voltage. And 79XX series is for Negative Voltage Regulators.

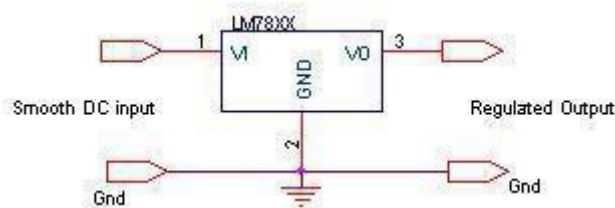


fig 3.13

After filtering the rectifier output the signal is given to a voltage regulator. The maximum input voltage that can be applied at the input is 35V. Normally there is a 2-3 Volts drop across the regulator so the input voltage should be at least 2-3 Volts higher than the output voltage. If the input voltage gets below the V_{min} of the regulator due to the ripple voltage or due to any other reason the voltage regulator will not be able to produce the correct regulated voltage.

Circuit diagram:

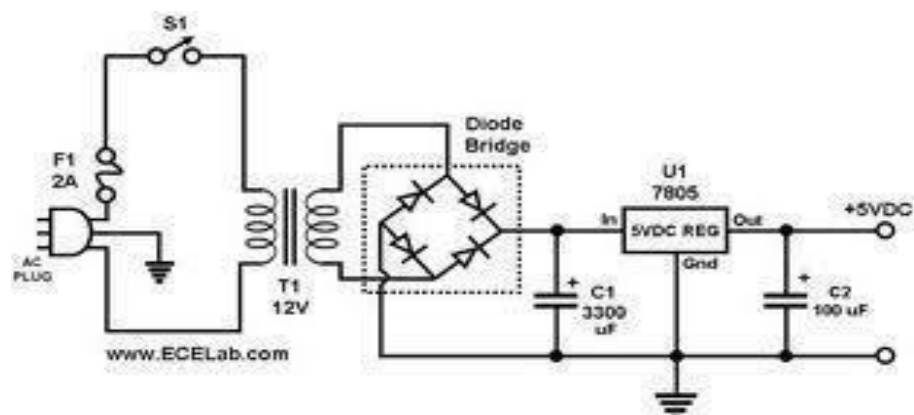


Fig 3.14 Circuit Diagram of power supply**IC 7805:**

7805 is an integrated three-terminal positive fixed linear voltage regulator. It supports an input voltage of 10 volts to 35 volts and output voltage of 5 volts. It has a current rating of 1 amp although lower current models are available. Its output voltage is fixed at 5.0V. The 7805 also has a built-in current limiter as a safety feature. 7805 is manufactured by many companies, including National Semiconductors and Fairchild Semiconductors.

The 7805 will automatically reduce output current if it gets too hot. The last two digits represent the voltage; for instance, the 7812 is a 12-volt regulator. The 78xx series of regulators is designed to work in complement with the 79xx series of negative voltage regulators in systems that provide both positive and negative regulated voltages, since the 78xx series can't regulate negative voltages in such a system.

The 7805 & 78 is one of the most common and well-known of the 78xx series regulators, as its small component count and medium-power regulated 5V make it useful for powering TTL devices.

Table 3.15 Specifications of IC7805

SPECIFICATIONS	IC 7805
V_{out}	5V
$V_{in} - V_{out}$ Difference	5V - 20V
Operation Ambient Temp	0 - 125°C



3.3 RELAY MODULE:

3.3.1 TYPES OF RELAY

A relay is an electrically controllable switch widely used in industrial controls, automobiles and appliances.

The relay allows the isolation of two separate sections of a system with two different voltage sources i.e., a small amount of voltage/current on one side can handle a large amount of voltage/current on the other side but there is no chance that these two voltages mix up.

1. Single-Channel Relay Board

A 1-channel relay module contains only one relay and is used for switching a single AC or DC load. It is ideal for small automation tasks like controlling one light, fan, or motor.

2. Two-Channel Relay Board

This module has **two relays**, allowing control of two independent devices. It is commonly used in home automation projects where multiple appliances need switching.

3. Four-Channel Relay Board

A 4-channel relay module provides four relays in one board, making it suitable for controlling multiple loads simultaneously. It is widely used in automation systems, robotics, and smart home panels.

4. Eight-Channel Relay Board

This module has **eight relays** and is used for large automation setups where multiple devices or circuits must be controlled from a single microcontroller. It is suitable for industrial and complex home automation projects.

5. Solid-State Relay (SSR) Board

An SSR board uses electronic components instead of mechanical contacts to switch loads. It operates silently, supports high switching speeds, and is ideal for industrial, sensitive, or noise-free applications.

6. Optocoupler/Opto-Isolated Relay Board

These relay boards include optocouplers for electrical isolation between the control circuit (microcontroller) and the load. This protects the microcontroller from high voltage spikes and improves safety.

3.3.2 Working Of Relay:

A photodiode is a semiconductor diode that functions as a photo detector. Photodiodes are packaged with either a window or optical fiber connection, to let in the light to the sensitive part of the device. They may also be used without a window to detect vacuum UV or X-rays.

A phototransistor is in essence nothing more than a bipolar transistor that is encased in a transparent case so that light can reach the base-collector junction. The phototransistor works like a photodiode, but with a much higher responsivity for light, because the electrons that are generated by photons in the base-collector junction are injected into the base, and this current is then amplified by the transistor operation.

Inductor

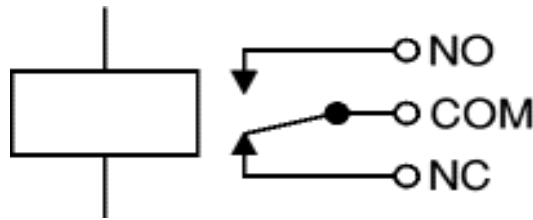


Fig 3.15: Circuit symbol of a relay

Principle of operation:

When current flows through the coil, a magnetic field is created around the coil i.e., the coil is energized. This causes the armature to be attracted to the coil. The armature's contact acts like a switch and closes or opens the circuit. When the coil is not energized, a spring pulls the armature to its normal state of open or closed. There are all types of relays for all kinds of applications.

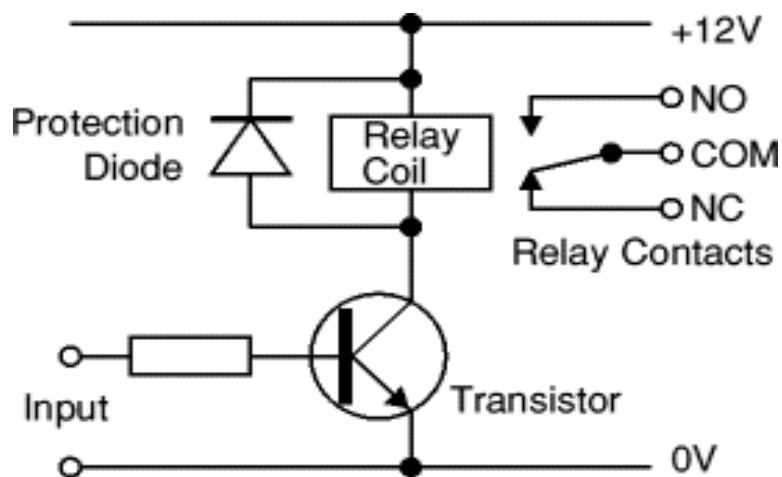


Fig 3.16: Relay Operation and use of protection diodes

Transistors and ICs must be protected from the brief high voltage 'spike' produced when the relay coil is switched off. The above diagram shows how a signal diode (eg 1N4148) is connected across the relay coil to provide this protection. The diode is connected 'backwards' so that it will normally not conduct. Conduction occurs only when the relay coil is switched off, at this moment the current tries to flow continuously through the coil and it is safely diverted through the diode. Without the diode no current could flow and the coil would produce a damaging high voltage 'spike' in its attempt to keep the current flowing.

In choosing a relay, the following characteristics need to be considered:

1. The contacts can be normally open (NO) or normally closed (NC). In the NC type, the contacts are closed when the coil is not energized. In the NO type, the contacts are closed when the coil is energized.
2. There can be one or more contacts. i.e., different types like SPST (single pole single throw), SPDT (single pole double throw) and DPDT (double pole double throw) relays.
3. The voltage and current required to energize the coil. The voltage can vary from a few volts to 50 volts, while the current can be from a few milliamps to 20milliamps. The relay has a minimum voltage, below which the coil will not be energized. This minimum voltage is called the “pull-in” voltage.
4. The minimum DC/AC voltage and current that can be handled by the contacts. This is in the range of a few volts to hundreds of volts, while the current can be from a few amps to 40A or more, depending on the relay.

DRIVING A RELAY:

An SPDT relay consists of five pins, two for the magnetic coil, one as the common terminal and the last pins as normally connected pin and normally closed pin. When the current flows through this coil, the coil gets energized. Initially when the coil is not energized, there will be a connection between the common terminal and normally closed pin. But when the coil is energized, this connection breaks and a new connection between the common terminal and normally open pin will be established. Thus

when there is an input from the microcontroller to the relay, the relay will be switched on. Thus when the relay is on, it can drive the loads connected between the common terminal and normally open pin. Therefore, the relay takes 5V from the microcontroller and drives the loads which consume high currents. Thus the relay acts as an isolation device.

Digital systems and microcontroller pins lack sufficient current to drive the relay. While the relay's coil needs around 10milli amps to be energized, the microcontroller's pin can provide a maximum of 1-2milli amps current. For this reason, a driver such as ULN2003 or a power transistor is placed in between the microcontroller and the relay. In order to operate more than one relay, ULN2003 can be connected between relay and microcontroller

3.3.3 Features:

1. Electrical Isolation

Most relay boards include **optocouplers**, which provide isolation between the low-voltage control circuit (microcontroller) and the high-voltage load, ensuring safety.

2. High-Voltage Switching Capability

Relay boards can control **AC or DC loads**, typically supporting 230V AC or 30V DC, allowing them to switch household appliances, lights, fans, and motors.

3. Multiple Channels

Available in different configurations like **1-channel, 2-channel, 4-channel, and 8-channel**, allowing control of multiple devices from a single controller.

4. Low Control Voltage

The relay coil is triggered using **3.3V or 5V**, making it compatible with microcontrollers like ESP32, Arduino, Raspberry Pi, etc.

5. LED Status Indicators

Each relay typically has an **LED indicator** that shows whether the relay is ON or OFF, making debugging easier.

6. Screw Terminals for Easy Wiring

Relay boards provide screw-type terminals for securely connecting external loads and power wires.

7. NO/NC/COM Terminals

Each relay supports:

- **NO (Normally Open)**
- **NC (Normally Closed)**
- **COM (Common)**

These allow flexible wiring depending on whether the load should be normally ON or OFF.

8. Mechanical or Solid-State Switching

Relay boards are available in:

- **Mechanical relays** (clicking sound),
- **Solid-State Relays (SSR)** (silent, fast switching).

9. Protection Components

Includes components like **flyback diodes**, **transistors**, and **resistors** to protect the microcontroller and ensure reliable switching.

10. Long-life Operation

Designed for thousands of switching cycles, ensuring durable performance in automation systems.

3.3.4 Applications:

1. Home Automation

Used to control home appliances such as lights, fans, AC units, water pumps, and heaters using microcontrollers like ESP32 or Arduino.

2. Industrial Automation

Relay boards are widely used to operate industrial machines, conveyor belts, motors, solenoids, and heavy electrical loads safely.

3. Remote Switching Systems

Allows appliances to be turned ON/OFF remotely through Wi-Fi, Bluetooth, or IoT platforms (e.g., Blynk, MQTT, Node-RED).

4. Smart Energy Management

Used in smart meters, automatic power cutoff systems, and energy-saving automation to control electrical loads based on usage or schedule.

5. Security Systems

Relay boards control alarms, sirens, electromagnetic door locks, and access control systems.

6. Automotive Applications

Used in vehicles to switch headlights, horns, fuel pumps, and other electrical components.

7. Robotics

Helps control high-power motors, actuators, and external devices that microcontrollers cannot drive directly.

8. HVAC Control

Used for controlling heating, ventilation, ventilation dampers, and air conditioning units.

9. Agriculture Automation

Controls irrigation pumps, valves, sprayers, and greenhouse environmental control systems.

10. Smart Switch Boards

Relay modules are used inside IoT-based switchboards to automate traditional electrical switches.

3.4 L293D MOTOR DRIVER

The MQ3 alcohol sensor is one of the series of MQ gas sensors, which can detect and monitor the alcohol gas present in the atmosphere. It is capable of detecting 25-500ppm alcohol gas concentration in the air. This article gives a brief description of the pin configuration, specifications, and Arduino interfacing of the MQ3 alcohol sensor. The alternatives of MQ3 alcohol sensors are MQ138 (benzene, hydrogen, alcohol, propane, toluene, formaldehyde gas), MQ303A (ethanol, smoke, and alcohol), MQ2(methane, smoke, LPG, butane), MQ214 (methane), MQ5 (natural gas and LPG), and MQ306A (LPG and butane).

3.4.1 Pin Diagram of L293D

The pin configuration/pin diagram is shown in the figure below.

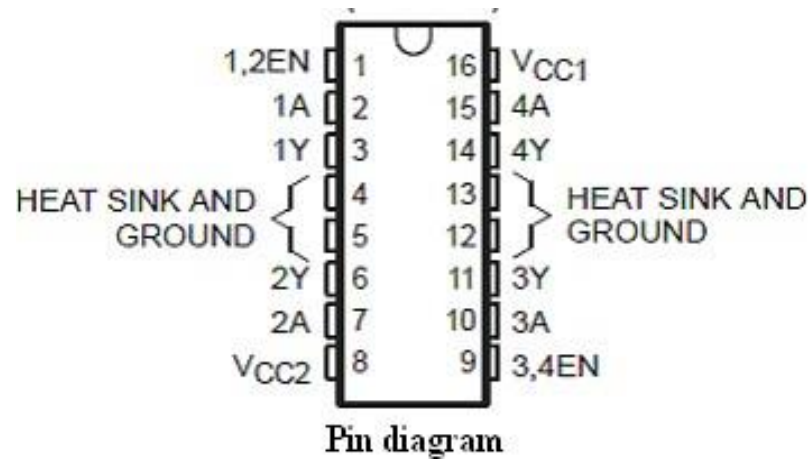


FIG 3.17

The L293 and L293D are quadruple high-current half-H drivers. The L293 is designed to provide bidirectional drive currents of up to 1 A at voltages from 4.5 V to 36 V. The L293D is designed to provide bidirectional drive currents of up to 600-mA at voltages from 4.5 V to 36 V. Both devices are designed to drive inductive loads such as relays, solenoids, dc and bipolar stepping motors, as well as other high-current/high-voltage loads in positive-supply applications.

All inputs are TTL compatible. Each output is a complete totem-pole drive circuit, with a Darlington transistor sink and a pseudo- Darlington source. Drivers are enabled in pairs, with drivers 1 and 2 enabled by 1,2EN and drivers 3 and 4 enabled by 3,4EN. When an enable input is high, the associated drivers are enabled and their outputs are active and in phase with their inputs. When the enable input is low, those drivers are disabled and their outputs are off and in the high-impedance state. With the proper data inputs, each pair of drivers forms a full-H (or bridge) reversible drive suitable for solenoid or motor applications. On the L293, external high-speed output clamp diodes should be used for inductive transient suppression.

A VCC1 terminal, separate from VCC2, is provided for the logic inputs to minimize device power dissipation. The L293 and L293D are characterized for operation from 0 to 70 degree Celsius.

3.4.2 Circuit Diagram of L293D:

Now, let's have a look at **how to interface the MQ3 alcohol sensor with the Arduino**

UNO. In this case, the analogue output pin of the MQ3 sensor is used for proper sensor calibration. The circuit diagram/how to use the MQ3 alcohol sensor with the Arduino is shown in the figure below.

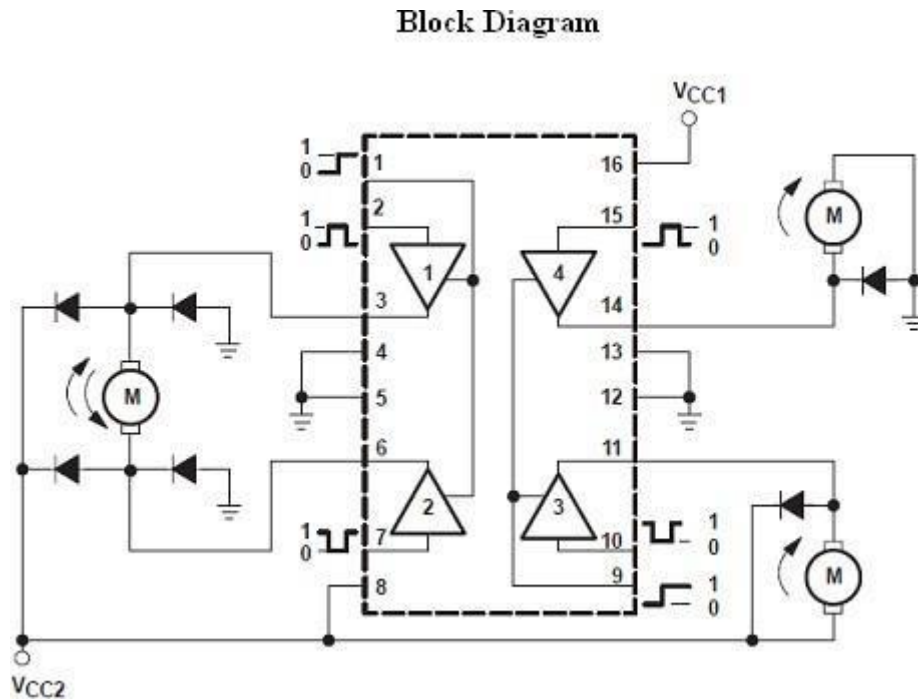


Fig 3.18 BLOCK/CIRCUIT DIAGRAM

The **L293D** is a dual H-bridge motor driver IC used to control the direction and speed of DC motors. It allows a low-power microcontroller (like ESP32, Arduino, etc.) to drive high-current motors safely. The IC contains **two H-bridge circuits**, which means it can independently control **two DC motors**.

In the block diagram, each H-bridge is built using pairs of transistors arranged in a cross-connected form. These transistors receive logic signals from the microcontroller through **input pins (IN1, IN2, IN3, IN4)**. Depending on the input combinations, the H-bridge switches ON/OFF different transistors to control the motor direction. When $IN1 = 1$ and $IN2 = 0$, current flows in one direction and the motor rotates clockwise. When $IN1 = 0$ and $IN2 = 1$, current reverses and the motor rotates anticlockwise. If both inputs are 0 or both are 1, the motor stops.

The IC has **two power supply pins**:

- **Vcc1** (5V) → powers the internal logic circuit.
- **Vcc2** (6–36V) → powers the motor load.

Diodes are included in the internal structure to protect the IC from back-EMF generated by the motors.

The **Enable pins (EN1, EN2)** activate or deactivate the H-bridges, allowing PWM signals to control motor speed.

Overall, the L293D provides safe and reliable motor control by isolating the high-current motor load from the low-power microcontroller, while enabling easy direction and speed control.

3.4.3 Applications:

The **applications of the L293D** are given below,

- Used to drive DC motors in robotics.
- Controls wheels in smart cars and mobile robots.
- Operates motors in home automation systems.
- Drives conveyor belt motors in small automation projects.
- Used in IoT-based motor control applications.
- Helps run actuators in mechatronics systems.
- Useful in educational and DIY electronic projects.

3.5 DC MOTOR

3.5.1 TYPES OF DC MOTOR

Description:

1. Brushed DC Motor

A simple motor that uses brushes and a commutator for switching current. It is inexpensive and commonly used in toys, small robots, and simple machines.

2. Brushless DC Motor (BLDC)

A motor that operates without brushes, using electronic control for commutation. It is more efficient, quieter, and used in drones, electric vehicles, and high-performance applications.

3. Permanent Magnet DC Motor (PMDC)

Uses permanent magnets instead of field windings. It offers good torque and is used in wipers, fans, pumps, and small automation devices.

4. Series-Wound DC Motor

The field winding and armature are connected in series, giving very high starting torque. Used in traction systems, cranes, and heavy machinery.

5. Shunt-Wound DC Motor

Has the field winding in parallel with the armature, giving stable speed. Commonly used in lathes, fans, and industrial machines.

6. Compound DC Motor

Combines series and shunt windings, offering both high torque and good speed regulation. Used in elevators, presses, and rolling mills.

7. Gear DC Motor

Attached with a gearbox to reduce speed and increase torque. Widely used in robotics, powered mechanisms, and automation projects.

3.5.2 WORKING OF DC MOTOR

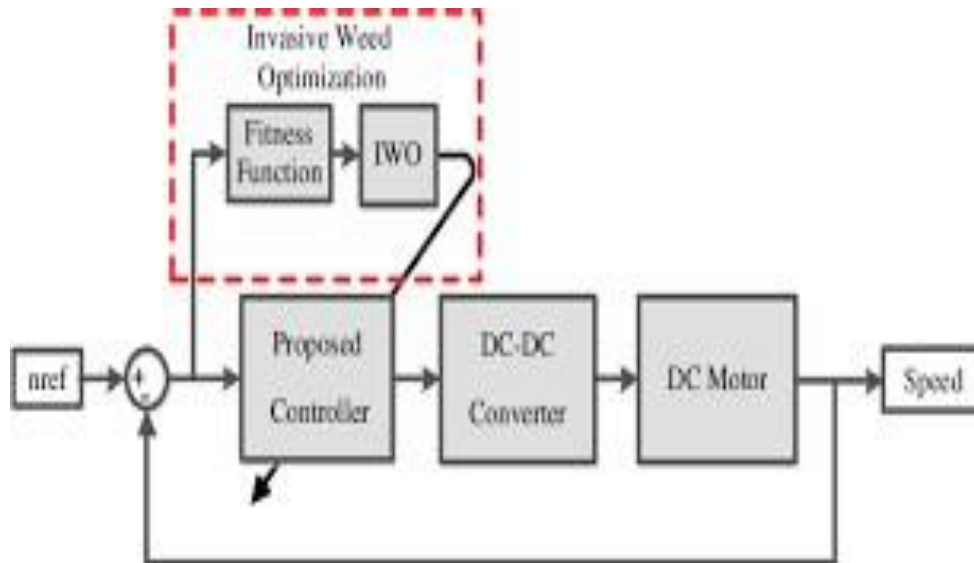


Fig 3.19: block Diagram of DC Motor

A **DC motor** converts **electrical energy** (direct current) into **mechanical rotational energy**. Its operation is based on the principle that:

When a current-carrying conductor is placed in a magnetic field, it experiences a mechanical force.

Main Components

1. Stator (Magnetic Field Source)

Provides a stationary magnetic field using permanent magnets or field windings.

Rotor / Armature

A rotating coil where current flows; it experiences force and begins to rotate.

3. Commutator

A mechanical switch that reverses the direction of current in the armature each half-turn, ensuring continuous rotation.

4. Brushes

Carbon contacts that allow current to flow from the external circuit into the rotating commutator.

Step-by-Step Working

1. Input Electrical Power

The DC motor receives a controlled DC voltage from a **DC–DC converter** (as shown in your diagram). This voltage determines the armature current.

2. Creation of Magnetic Field

The stator produces a constant magnetic field.

3. Armature Current Generates Force

When current flows through the armature winding:

- The magnetic field interacts with the armature current.
- This produces a **Lorentz force** on the armature conductors.
- The forces combine to generate a **torque** on the rotor.

4. Commutation Ensures Continuous Rotation

As the rotor turns, the commutator reverses current direction at the right moment so torque always acts in the same rotational direction.

5. Rotor Spins → Produces Mechanical Output

This rotational speed is sent to the **load** (in your diagram: *Speed output*).

3.5.3 ADVANTAGES

1. Simple and Accurate Speed Control

DC motors allow **easy and precise control of speed** over a wide range by simply adjusting the supply voltage or field current.

This makes them ideal for applications requiring **variable speed**.

2. High Starting Torque

They deliver **high starting torque**, which makes them suitable for applications like electric vehicles, cranes, elevators, and hoists.

3. Quick Response to Load Changes

DC motors respond fast to changes in load and control signals, ensuring **stable and smooth operation**.

4. Easy to Control Using Power Electronics

Modern controllers and DC–DC converters make it simple to integrate DC motors into automated systems.

5. Good Speed Regulation

They maintain nearly constant speed even with varying loads, especially **shunt and permanent- magnet DC motors**.

6. Smooth and Steady Motion

DC motors offer **smooth rotation**, which is especially useful in robotics and precision motion control.

7. High Efficiency

They have good energy efficiency due to low electrical and mechanical losses.

8. Simple Construction & Maintenance

Especially **brushed DC motors** have simple construction, making them easy to repair and maintain.

9. Wide Range of Power Ratings

DC motors are available from fractional horsepower up to high-power industrial motors.

10. Reliable for Low-Speed Applications

They provide high torque even at **low speeds**, making them ideal for traction systems and conveyors.

3.6 BATTERY

Definition:

A **3.7 V Lithium-ion battery** is a rechargeable electrochemical energy storage device that provides a **nominal voltage of 3.7 volts**. It uses **lithium-based chemical reactions** to store and release electrical energy efficiently. These batteries are widely used in portable electronics due to their **high energy density, lightweight construction, and long cycle life**.

3.6.1 WORKING

The working of a 3.7 V Li-ion battery is based on the **movement of lithium ions** between two electrodes:

Charging

- Lithium ions move from the **positive electrode (cathode)** to the **negative electrode (anode)**.
- Electrical energy is stored as chemical energy.

Discharging

- When connected to a load, lithium ions move back from the **anode** to the **cathode**.
- This movement generates a flow of electrons through the external circuit → **providing power**.

The chemical reactions are reversible, which makes the battery **rechargeable**.



FIG 3.20 3.7V BATTERY

3.6.2 SPECIFICATIONS

Parameter	Typical Value
Nominal Voltage	3.7 V
Charging Voltage	4.2 V
Capacity	1000–3000 mAh (example: 1200 mAh)
Battery Type	Lithium-ion / Li-Po
Parameter	Typical Value
Chemistry	LiCoO ₂ , LiPo, INR

	(varies by model)
Energy Density	High (150–250 Wh/kg)
Cycle Life	300–500 cycles
Operating Temperature	0°C to 45°C (charging)
Discharge Temperature	–20°C to 60°C
Charging Method	CC-CV (Constant Current → Constant Voltage)
Shape	Cylindrical (18650) or pouch type

3.6.3 APPLICATIONS

3.7 V Li-ion batteries are commonly used in:

- Mobile phones and smartphones
- IoT devices and embedded systems
- Arduino / ESP32 / Raspberry Pi projects
- Wearable devices
- Rechargeable toys and gadgets
- Power banks
- Bluetooth speakers and headphones
- Small robots and mini-vehicles
- Handheld tools and instruments

CHAPTER-4

SOFTWARE EXPLANATION

4.1 : Introduction

This project is implemented using following software's:

- Express PCB – for designing circuit
- Arduino IDE compiler - for compilation part
- Proteus 7 (Embedded C) – for simulation part

4.2 : IOT Interface

When a project is first started you will be greeted with a yellow outline. This yellow outline is the dimension of the PCB. Typically after positioning of parts and traces, move them to their final position and then crop the PCB to the correct size. However, in designing a board with a certain size constraint, crop the PCB to the correct size before starting. Fig: 4.1 show the toolbar in which the each button has the following functions:



Fig: 4.1 Tool bar necessary for the interface

The select tool: It is fairly obvious what this does. It allows you to move and manipulate parts. When this tool is selected the top toolbar will show buttons to move traces to the top / bottom copper layer, and rotate buttons.

The zoom to selection tool: does just that. **The place pad:** button allows you to place small solder pads which are useful for board connections or if a part is not in the part library but the part dimensions are available. When this tool is selected the top toolbar will give you a large selection of round holes, square holes and surface mount pads

The place component: tool allows you to select a component from the top toolbar and then by clicking in the workspace places that component in the orientation chosen using the buttons next to the component list. The components can always be rotated afterwards with the select tool if the orientation is wrong.

The place trace: tool allows you to place a solid trace on the board of varying thicknesses. The top toolbar allows you to select the top or bottom layer to place the trace on.

The Insert Corner in trace: button does exactly what it says. When this tool is selected, clicking on a trace will insert a corner which can be moved to route around components and other traces.

When making a PCB you have the option of making a single sided board, or a double sided board. Single sided boards are cheaper to produce and easier to etch, but much harder to design for large projects. If a lot of parts are being used in a small space it may be difficult to make a single sided board without jumpering over traces with a cable. While there's technically nothing wrong with this, it should be avoided if the signal travelling over the traces is sensitive (e.g. audio signals).

The place pad: button allows you to place small soldier pads which are useful for board connections or if a part is not in the part library but the part dimensions are available. When this tool is selected the top toolbar will give you a large selection of round holes, square holes and surface mount pads

The place component: tool allows you to select a component from the top toolbar and then by clicking in the workspace places that component in the orientation chosen using the buttons next to the component list. The components can always be rotated afterwards with the select tool if the orientation is wrong.

The place trace: tool allows you to place a solid trace on the board of varying thicknesses. The top toolbar allows you to select the top or bottom layer to place the trace on.

The Insert Corner in trace: button does exactly what it says. When this tool is selected, clicking on a trace will insert a corner which can be moved to route around components and other traces.

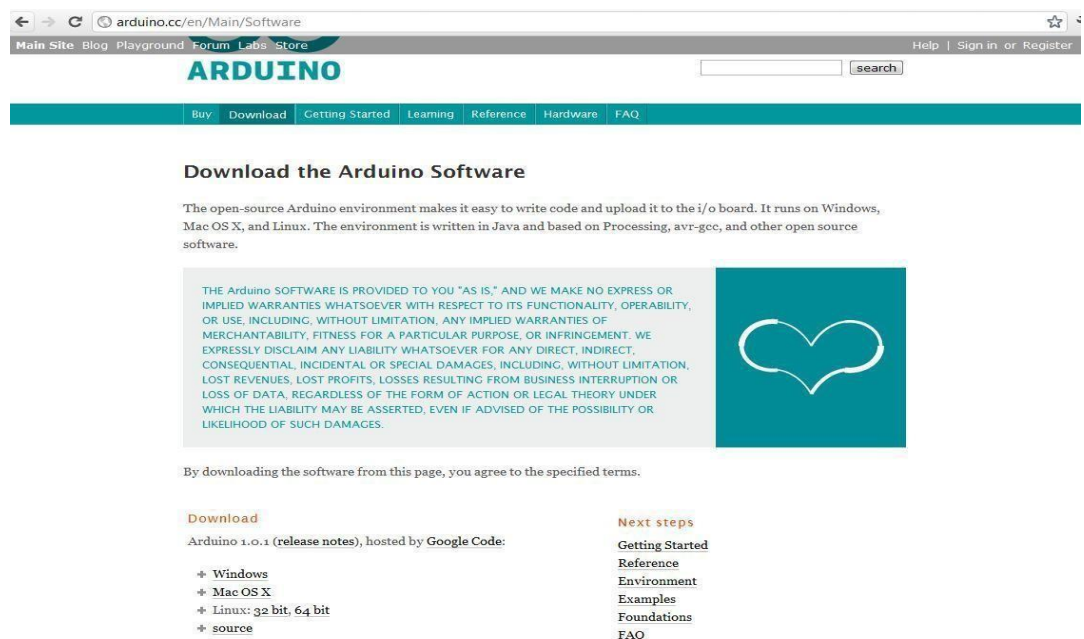
4.3 : Design Considerations

Before starting a project there are several ways to design a PCB and one must be chosen to suit the project's needs. Single sided, or double sided?

When making a PCB you have the option of making a single sided board, or a double sided board. Single sided boards are cheaper to produce and easier to etch, but much harder to design for large projects. If a lot of parts are being used in a small space it may be difficult to make a single sided board without jumpering over traces with a cable. While there's technically nothing wrong with this, it should be avoided if the signal travelling over the traces is sensitive (e.g. audio signals).

A double sided board is more expensive to produce professionally, more difficult to etch on a DIY board, but makes the layout of components a lot smaller and easier. It should be noted that if a trace is running on the top layer, check with the components to make sure you can get to its pins with a soldering iron. Large capacitors, relays, and similar parts which don't have axial leads can NOT have traces on top unless boards are plated professionally.

4.4 ESP32 PROGRAMMING USING ARDUINO IDE



arduino.cc/en/Main/Software

Main Site Blog Playground Forum Labs Store

ARDUINO

Help | Sign in or Register

Buy Download Getting Started Learning Reference Hardware FAQ

Download the Arduino Software

The open-source Arduino environment makes it easy to write code and upload it to the i/o board. It runs on Windows, Mac OS X, and Linux. The environment is written in Java and based on Processing, avr-gcc, and other open source software.

THE ARDUINO SOFTWARE IS PROVIDED TO YOU "AS IS," AND WE MAKE NO EXPRESS OR IMPLIED WARRANTIES WHATSOEVER WITH RESPECT TO ITS FUNCTIONALITY, OPERABILITY, OR USE, INCLUDING, WITHOUT LIMITATION, ANY IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, OR INFRINGEMENT. WE EXPRESSLY DISCLAIM ANY LIABILITY WHATSOEVER FOR ANY DIRECT, INDIRECT, CONSEQUENTIAL, INCIDENTAL OR SPECIAL DAMAGES, INCLUDING, WITHOUT LIMITATION, LOST REVENUES, LOST PROFITS, LOSSES RESULTING FROM BUSINESS INTERRUPTION OR LOSS OF DATA, REGARDLESS OF THE FORM OF ACTION OR LEGAL THEORY UNDER WHICH THE LIABILITY MAY BE ASSERTED, EVEN IF ADVISED OF THE POSSIBILITY OR LIKELIHOOD OF SUCH DAMAGES.

By downloading the software from this page, you agree to the specified terms.

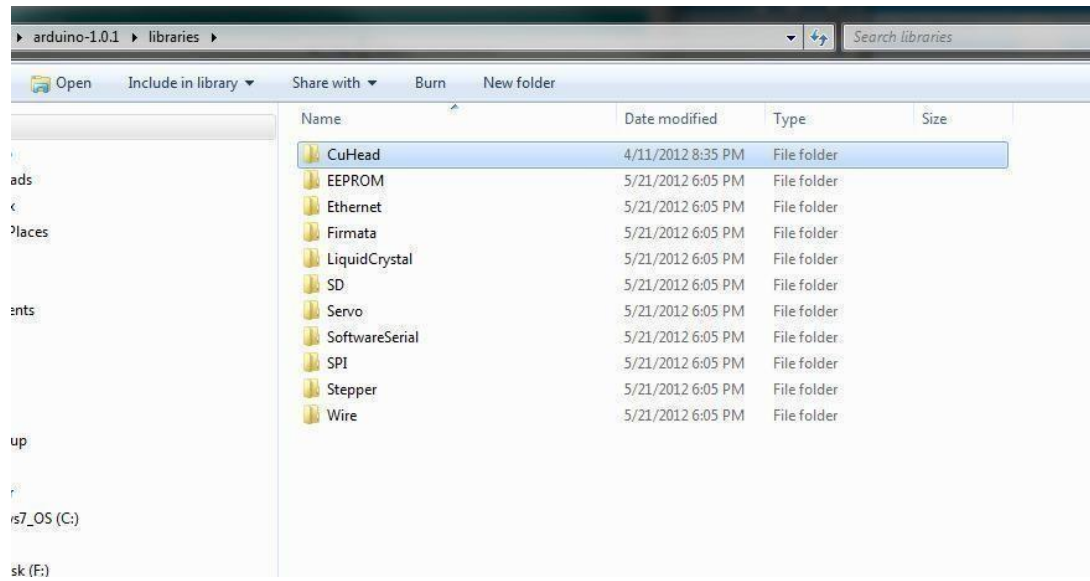
Download

Arduino 1.0.1 (release notes), hosted by Google Code:

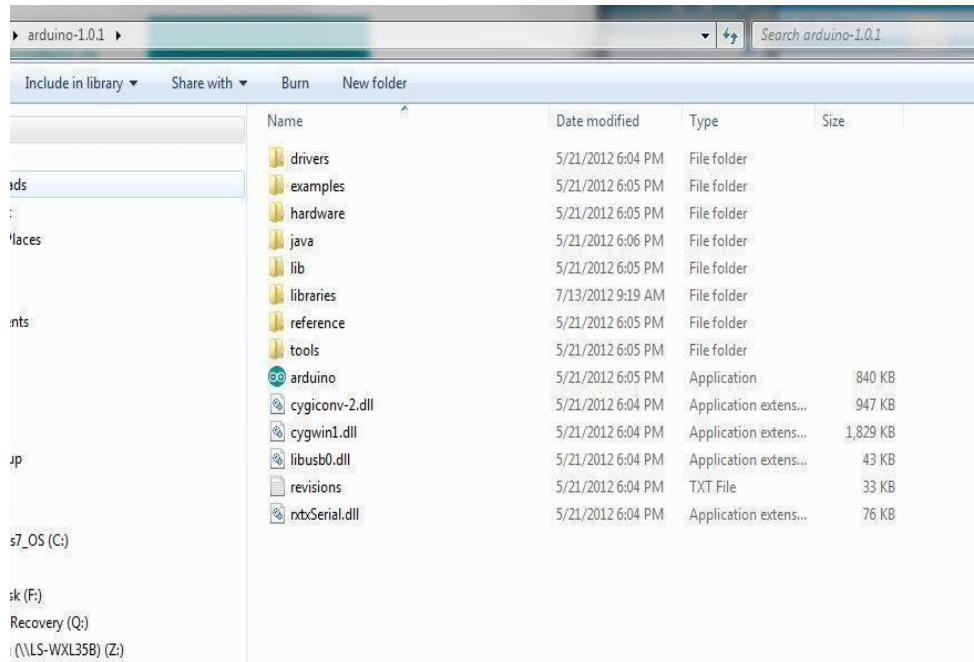
- ✦ Windows
- ✦ Mac OS X
- ✦ Linux: 32 bit, 64 bit
- ✦ source

Next steps

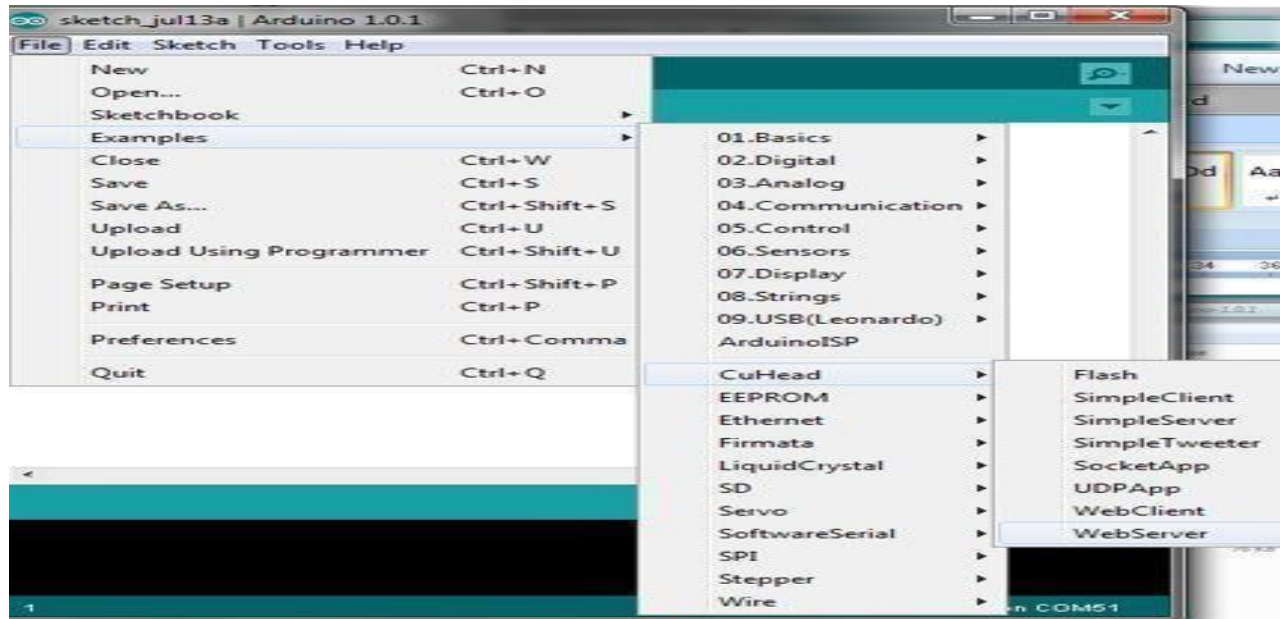
- Getting Started
- Reference
- Environment
- Examples
- Foundations
- FAQ



Launch Arduino IDE by double click “Arduino IDE” below



1. One example

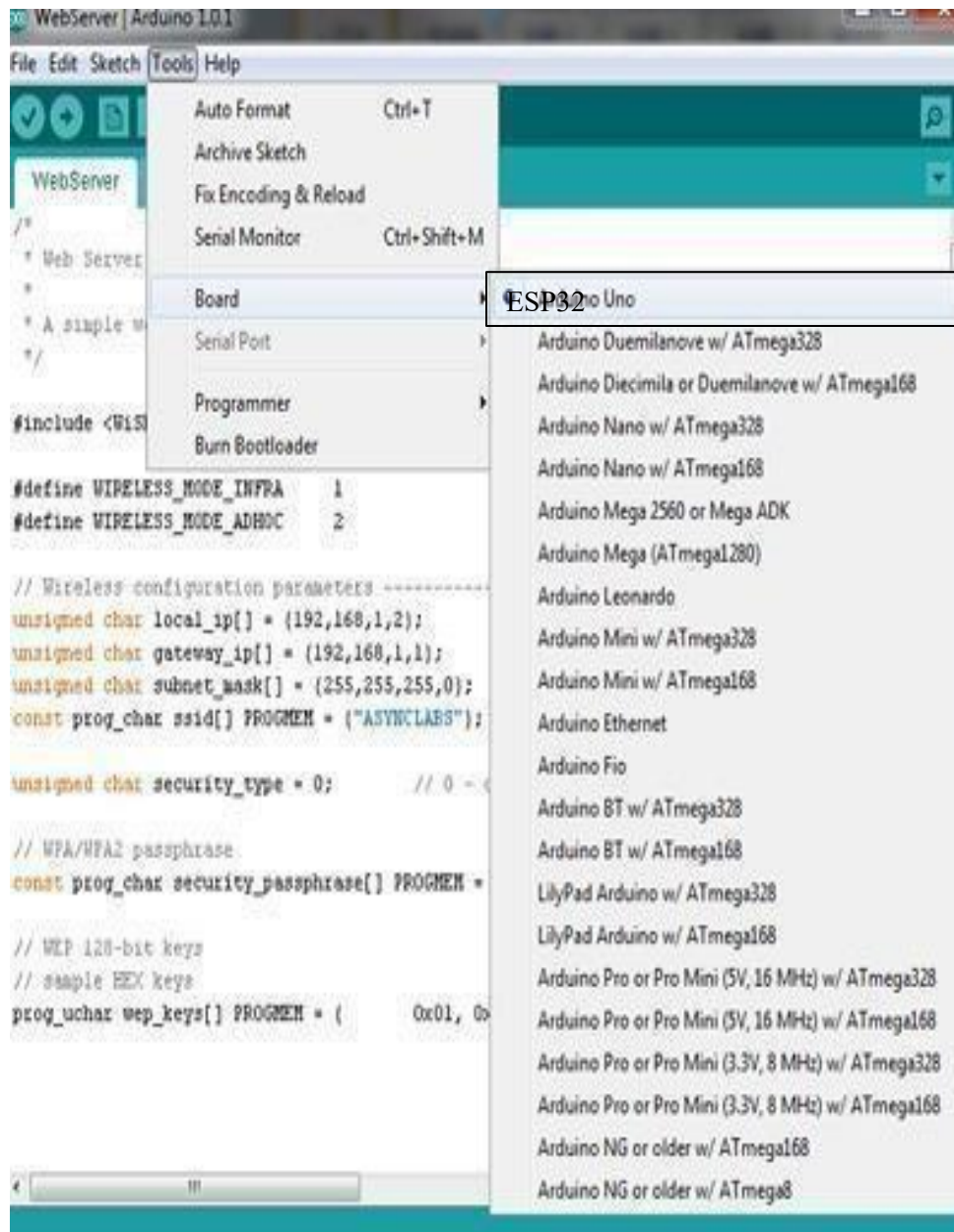


REALTIME PROJECT

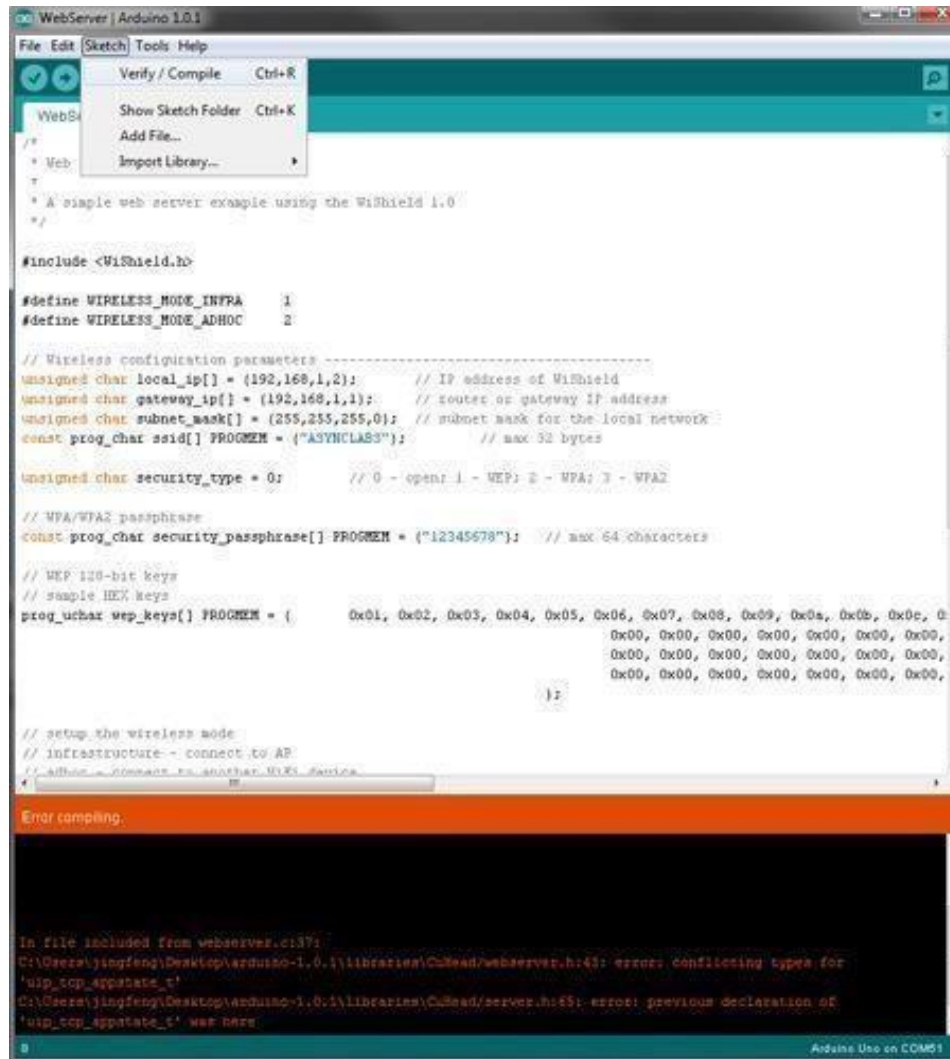
DEPT OF ECE AITH HYD Page 45

HOME AUTOMATION USING IOT

2. Select the target board as “ESP32”:



3. Click Sketch-> Verify/Compile:



CHAPTER- 5

PROGRAMMING OF DEVICE

```
#include<string.h>

#include <WiFiClientSecure.h> // Include the HTTPS library

#include "Arduino.h"

#include <EMailSender.h>


#define BLYNK_TEMPLATE_ID "TMPL390hXLvCC"

#define BLYNK_TEMPLATE_NAME "HOME AUTOMATION"

#define BLYNK_AUTH_TOKEN
"9b2VKbkFYoafDZgxTmTEcpoBANUs3mjo"


/*

#define BLYNK_TEMPLATE_ID "TMPL3aXWPJo1Z"

#define BLYNK_TEMPLATE_NAME "IOT PROJECT"

#define BLYNK_AUTH_TOKEN
"reRdpaDxKpk4BeRwsuFdWPdCUHK4LqFu"

*/


int val=0; int
val1=0;  int
force=16; int
```

```
l1=26;//25
int
l2=25;//26
int
l3=33;//32
int
pp=32;//33
#define BLYNK_PRINT Serial

#if defined(ESP8266)
#include <ESP8266WiFi.h>
#include <BlynkSimpleEsp8266.h>
#include <WiFiClientSecure.h> // Include the HTTPS library
#include <ESP8266WiFi.h>      // Include the Wi-Fi library
#include <ESP8266WiFiMulti.h> // Include the Wi-Fi-Multi library
ESP8266WiFiMulti wifiMulti;  // Create an instance of the
ESP8266WiFiMulti class, called 'wifiMulti'
#elif defined(ESP32)
#include <WiFi.h>
#include <WiFiClient.h>
#include <BlynkSimpleEsp32.h>
#include <WiFiMulti.h>
```

```
WiFiMulti wifiMulti; // Create an instance of the ESP8266WiFiMulti class, called  
'wifiMulti'
```

```
#endif  
char auth[] = BLYNK_AUTH_TOKEN;
```

```
// Your WiFi credentials.  
  
// Set password to "" for open networks.  
  
char ssid[] = "ZTE-sUQdqa";  
char pass[] = "5hjgxyh9";
```

```
uint8_t connection_state = 0; uint16_t  
reconnect_interval = 10000;  
  
WiFiClient client;  
  
String data1="";  
String data2="cmd"; String  
data=" MESSAGE";  
  
float latitude=18.068934633390842; //18.068934633390842,  
78.85471331292823  
  
float logitude=78.85471331292823; //78.38541332339298  
  
  
String location="
```

```
http://maps.google.com/maps?&z=15&mrt=yp&t=k&q="+String(latitude,6)+"+"
"+String(logitude,6);
```

```
BlynkTimer timer;
// This function sends Arduino's up time every second to Virtual Pin (5). // In
the app, Widget's reading frequency should be set to PUSH. This means
// that you define how often to send data to Blynk App.
```

```
EMailSender emailSend("sivaprasadpilla7@gmail.com", "tvgcjlhvnptetvvq"); void
gmail()
{
  EMailSender::EmailMessage message;

  message.subject = "MAIL FROM IOT MODULE: : "+data1; message.message
  = "MAIL FROM IOT MODULE: : "+data1;

  EMailSender::Response resp = emailSend.send("siva9014475682@gmail.com",
message);

  Serial.println("Sending status: ");

  Serial.println(resp.status);
  Serial.println(resp.code);
  Serial.println(resp.desc);
}
```

```
void upload()
{
const char* server4 = "api.thingspeak.com"; const
char* _getLink4 =
"https://api.thingspeak.com/update?api_key=PWX6Z0ZH2CQXFG3X&field1="
"; // Thingspeak.com
//const char* _getLink4 =
"https://api.thingspeak.com/update?api_key=8QEMT0G004GKO1L8&field1=";
// Thingspeak.com

// Serial.println("data uploading");delay(1000); client.connect(server4,80);
if (client.connect(server4,80)) // "184.106.153.149" or api.thingspeak.com
https://api.thingspeak.com/apps/thinghttp/send_request?api_key=CT9B331KB5
PLM1G5
{
String getStr4 = _getLink4;
client.print("GET "+getStr4+data1+"\n");
client.print("HTTP/1.1\n");
client.print("Host: api.thingspeak.com\n"); client.print("Connection:
close\n\n\n");
```

```
}  
  
client.stop();  
  
}  
  
void myTimerEvent()  
{ back:  
  while(Serial.available())  
  {  
    data1=Serial.readString();  
    Blynk.virtualWrite(V5,data1);delay(1000);  
    Blynk.virtualWrite(V6,data1);delay(1000);  
    Blynk.virtualWrite(V7,data1);delay(1000);  
    Blynk.virtualWrite(V8,data1);delay(1000);  
    Blynk.virtualWrite(V11,data1);delay(1000);  
    //data1.replace(" ","_");delay(1000);  
    //data1.replace(":","_");delay(1000);  
    //upload();delay(1000);  
    //data1=data1+location;delay(1000);  
    //gmail();delay(1000);  
  }  
}
```

```
/* if (Serial.available() > 0)
{ // Read the data from
  Arduino

  String dataFromArduino = Serial.readStringUntil('\n');

  int sensor1Value, sensor2Value, sensor3Value, sensor4Value;

  // Parse and extract individual sensor values
  sscanf(dataFromArduino.c_str(), "Sensor1: %d, Sensor2: %d, Sensor3: %d",
    &sensor1Value, &sensor2Value, &sensor3Value);

  Serial.print("Sensor1: ");
  Serial.print(sensor1Value);
  Serial.print(", Sensor2: ");
  Serial.print(sensor2Value);
  Serial.print(", Sensor3: ");
  Serial.println(sensor3Value);

  Blynk.virtualWrite(V4, "\r\nsensor1Value:");Blynk.virtualWrite(V4,sensor1Value);
  Blynk.virtualWrite(V4, "\r\nsensor2Value:");Blynk.virtualWrite(V4,sensor2Value);
  Blynk.virtualWrite(V4, "\r\nsensor3: Value");Blynk.virtualWrite(V4,sensor3Value); }
*/
```

```
}
```

```
BLYNK_WRITE(V0)
```

```
{ int pinValue = param.asInt(); // assigning incoming value from pin V1 to a
variable
```

```
//Serial.print("VALUE V0:");//Serial.println(String(pinValue)); if(pinValue==0)
```

```
{
```

```
Serial.print("1");delay(100);
```

```
Blynk.virtualWrite(V4, "Load:1 On\r\n");
```

```
digitalWrite(11,LOW);delay(1000);
```

```
} if(pinValue==1)
```

```
{
```

```
Serial.print("2");delay(100);
```

```
Blynk.virtualWrite(V4, "Load:1 Off\r\n");
```

```
digitalWrite(11,HIGH);delay(1000);
```

```
}
```

```
}
```

```
BLYNK_WRITE(V1)
```

```
{ int pinValue = param.asInt(); // assigning incoming value from pin V1 to a
variable
```

```

//Serial.print("VALUE V1:");Serial.println(String(pinValue));

//lcd.clear();lcd.print("LIGHT1:");lcd.print(String(pinValue));delay(1000);

if(pinValue==0)
{
Serial.print("3");delay(100);
Blynk.virtualWrite(V4,                "Load:2                On\r\n");
digitalWrite(12,LOW);delay(1000);

} if(pinValue==1)
{
Serial.print("4");delay(100);
Blynk.virtualWrite(V4,                "Load:2                Off\r\n");
digitalWrite(12,HIGH);delay(1000);

}

}

BLYNK_WRITE(V2)

{ int pinValue = param.asInt(); // assigning incoming value from pin V1 to a
variable

//Serial.print("VALUE V2:");Serial.println(String(pinValue)); if(pinValue==0)

```

```

{
  Serial.print("5");delay(100);
  Blynk.virtualWrite(V4, "Load:3 On\r\n"); digitalWrite(l3,LOW);delay(1000);

} if(pinValue==1)
{
  Serial.print("6");delay(100);
  Blynk.virtualWrite(V4, "Load:3 Off\r\n");
  digitalWrite(l3,HIGH);delay(1000);

}
}

```

```

BLYNK_WRITE(V3)
{ int pinValue = param.asInt(); // assigning incoming value from pin V1 to a
variable
//Serial.print("VALUE V3:");Serial.println(String(pinValue)); if(pinValue==0)
{
  Serial.print("7");delay(100);
  Blynk.virtualWrite(V4, "Load:4 On\r\n");
  digitalWrite(pp,LOW);delay(1000);

} if(pinValue==1)
{

```

```

    Serial.print("8");delay(100);

    Blynk.virtualWrite(V4, "Load:4 Off\r\n"); digitalWrite(pp,HIGH);delay(1000);

}

}

```

```

BLYNK_WRITE(V11)

{ int pinValue = int(param.asInt()); // assigning incoming value from pin V1 to a
variable

Serial.print("DATA FROM TERMINAL:");Serial.println(pinValue);

//Blynk.virtualWrite(V4,"DATA FROM
TERMINAL:");Blynk.virtualWrite(V4,pinValue);
if(pinValue==12345)
{

Serial.println("CORRCET    PASSWORD    LOCKER    OPEN");delay(1000);
Blynk.virtualWrite(V4,"LOCKER OPEN");

}

else if(pinValue==54321)
{

```

```
Serial.println("CORRCET PASSWORD LOCKER CLOSE");delay(1000);  
Blynk.virtualWrite(V4,"LOCKER CLOSE");  
}  
}
```

```
/*#include <SimpleDHT.h> int  
pinDHT11 =13;  
SimpleDHT11 dht11(pinDHT11);  
*/
```

```
void senddata1()  
{  
/* byte temperature = 0; byte  
humidity = 0; int err =  
SimpleDHTErrSuccess;  
if((err = dht11.read(&temperature, &humidity, NULL)) != SimpleDHTErrSuccess)  
{  
// Serial.print("Read DHT11 failed, err=");  
Serial.print(SimpleDHTErrCode(err));  
// Serial.print(","); Serial.println(SimpleDHTErrDuration(err)); delay(1000); return;  
}
```

```
int temp=((int)temperature);
```

```

int hum=((int)humidity);delay(150);

int temp=touchRead(4);delay(10);
Serial.println("Touch Value:"+String(temp));delay(100); if(temp<30)
{ val=val+1;
Blynk.virtualWrite(V4,"\r\n"+String(val)+".Alert.Touch
Value:"+String(temp));delay(100);
} else
{ val=0;
} int
forceval=digitalRead(force);delay(10);
if(forceval==LOW)
{ val1=val1+1;
Blynk.virtualWrite(V4,"\r\n"+String(val1)+".Abusing          Alert.");delay(100);
Serial.println("Absuing Alert...");
} else
{
val1=0;
}
*/
}

```

```
void setup()
{
  pinMode(force,INPUT);

  pinMode(l1,OUTPUT);pinMode(l2,OUTPUT);pinMode(l3,OUTPUT);pinMode
  (pp,OUTPUT);

  digitalWrite(l1,HIGH);digitalWrite(l2,HIGH);digitalWrite(l3,HIGH);digitalWrit
  e(pp,HIGH);

  // Debug console
  Serial.begin(9600);          wifiMulti.addAP("consciencetechnologies",
  "484conscience777"); // add Wi-
  Fi networks you want to connect to
  wifiMulti.addAP("sivaji",      "sivaji.123");
  wifiMulti.addAP("ZTE-sUQdqa", "5hjpgxyh9");
  wifiMulti.addAP("project",     "project.123");
  wifiMulti.addAP("123456789", "123456789");

  //Serial.println("Connecting ...");

  while (wifiMulti.run() != WL_CONNECTED) { // Wait for the Wi-Fi to
  connect: scan for Wi-Fi networks, and connect to the strongest of the
  networks above delay(250);
```

```
    Serial.print('.');  
}  
  
Serial.println('\n');  
  
Serial.print("Connected to ");  
Serial.println(WiFi.SSID());           // Tell us what network we're connected to  
  
Serial.print("IP address:\t");  
  
  
//Serial.println(WiFi.localIP());       // Send the IP address of the ESP8266 to  
the computer  
  
//Serial.println('\n');  
  
Blynk.begin(auth, ssid, pass);delay(1000); //  
  
You can also specify server:  
  
//Blynk.begin(auth, ssid, pass, "blynk.cloud", 80);  
  
//Blynk.begin(auth,      ssid,      pass,      IPAddress(192,168,1,100),      8080);  
timer.setInterval(1000L, myTimerEvent);  
  
//timer.setInterval(500L, senddata1);  
}  
  
  
void loop()  
{  
  
  
  
  
  
  
  
  
    Blynk.run();  
  
    timer.run();  
  
}
```

```
// You can inject your own code or combine it with other sketches. //  
Check other examples on how to communicate with Blynk. Remember //  
to avoid delay() function!  
  
}
```

CHAPTER- 6

ADVANTAGES AND APPLICATIONS

Advantages:

- Easy remote control of home appliances
- Saves energy and reduces electricity bills
- Improves home security and safety
- Real-time monitoring of devices
- Increases comfort and convenience
- Automation through scheduling
- Alerts and notifications on phone
- Integration with other smart devices
- Helpful for elderly or disabled people
- Long-term cost savings

APPLICATIONS:

- Smart lighting control
- Smart door locks and security systems
- Home surveillance and CCTV monitoring
- Automated fans and AC temperature control
- Smart energy meter monitoring
- Fire, smoke, and gas leak detection

- Smart water tank level control
- Voice-controlled home appliances

CHAPTER-7

EXPLANATION OF RESULT

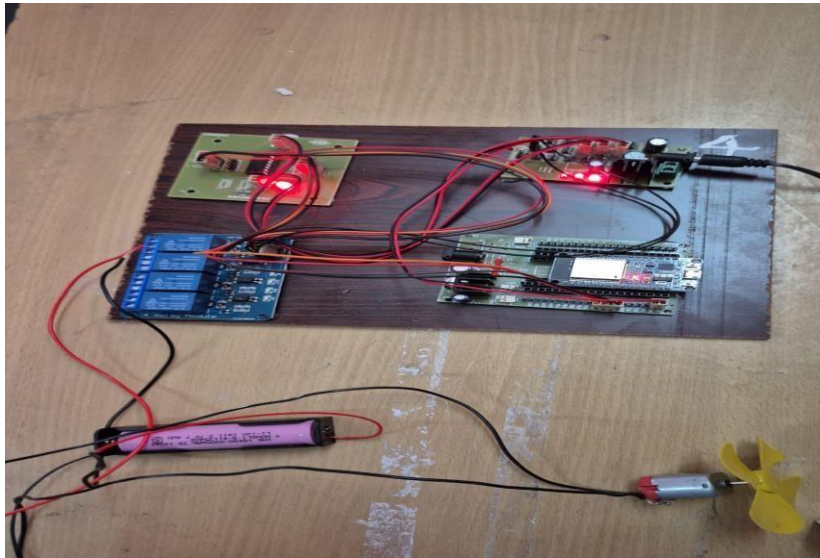
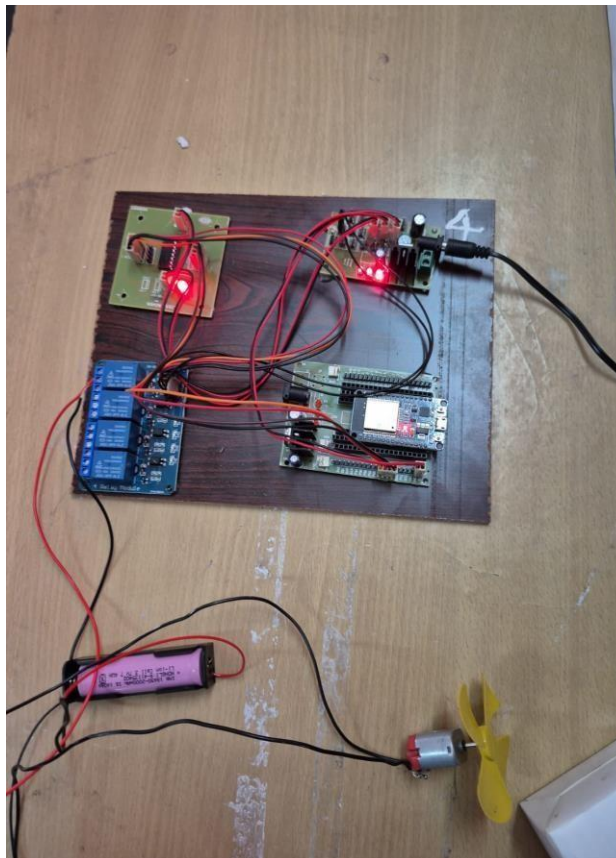


Fig 7.1 hardware kit

Fig.7.2 Working Model of Home Automation Using IoT



CHAPTER-8

CONCLUSION AND FUTURE SCOPE

CONCLUSION:

The IoT-based home automation system developed in this project successfully demonstrates how modern technology can be used to simplify and enhance everyday living. By utilizing the ESP32 microcontroller, the system provides reliable wireless communication, allowing users to remotely control and monitor home appliances with ease. This project improves convenience, increases safety through real-time alerts, and promotes energy efficiency by enabling intelligent device management. The design is cost-effective, scalable, and flexible, making it suitable for integration into various home environments. Overall, the project proves that IoT-based automation can transform traditional homes into smart, efficient, and user-friendly spaces, offering a strong foundation for future advancements.

FUTURE SCOPE:

- Integration with voice assistants (Alexa, Google Assistant, Siri)
- Adding AI for automatic decision-making based on user habits
- Expanding to full home security with cameras and motion detection
- Connecting more sensors (temperature, humidity, gas, fire, PIR)
- Creating a mobile app for better control and monitoring
- Adding battery backup or solar power for uninterrupted operation
- Implementing data analytics for energy usage patterns
- Using edge computing for faster and safer device control

CHAPTER-9

REFERENCES

1. Raj Kamal, *Internet of Things: Architecture and Design Principles*, McGraw Hill Education, 2017.

2. Arshdeep Bahga and Vijay Madisetti, *Internet of Things: A Hands-On Approach*, Universities Press, 2015.
3. Espressif Systems, *ESP32 Datasheet*, Available at: <https://www.espressif.com>
4. Arduino, *ESP32 Programming and Development Guide*, Available at: <https://www.arduino.cc>
5. Texas Instruments, *L293D Motor Driver IC Datasheet*, Available online.
6. ON Semiconductor, *7805 Voltage Regulator Datasheet*, Available online.
7. Songle Relay Company, *Relay Module (SRD-05VDC-SL-C) Datasheet*, Available online.
8. Electronics Hub, *Internet of Things (IoT) and ESP32 Tutorials*, Available at: <https://www.electronicshub.org>
9. Circuit Digest, *ESP32 Based IoT Projects and Tutorials*, Available at: <https://circuitdigest.com>
10. Wikipedia, *Internet of Things*, Available at: https://en.wikipedia.org/wiki/Internet_of_things
11. TutorialsPoint, *Internet of Things Tutorial*, Available at: <https://www.tutorialspoint.com>
12. YouTube, *ESP32 Home Automation Project Tutorials*, Various Channels.